

Graph-Preconditioned Estimation of High-Dimensional Fixed-Effect Models

Alexander Fischer and Kristof Schröder

Draft: May 2026

Abstract. The Method of Alternating Projections (MAP) is the standard algorithm for high-dimensional fixed-effect regressions. It is fast when absorbed factors are well connected, but converges slowly on sparse or nearly nested fixed-effect graphs, such as matched employer-employee panels where worker-firm mobility links separate worker and firm effects. The convergence behavior of MAP depends on the mobility pattern linking workers to firms, yet MAP only indirectly makes use of this information by iterating over one fixed effect at a time. That mobility pattern is, however, directly encoded in the matrix of worker-firm match counts, which after a sign change is a Laplacian and admits sparse approximate Cholesky factorization. We propose a graph-preconditioned Krylov solver with a reusable additive Schwarz preconditioner that exploits this structure through local factor-pair subproblems. Benchmarks show MAP remains fastest on dense designs, while graph preconditioning reduces runtime on sparse worker-firm graphs and under strong sorting.

Keywords: high-dimensional fixed effects; alternating projections; preconditioning; matched employer-employee data; computational econometrics

JEL codes: C55; C63; C81; C87; J31

1. Introduction

Fixed-effects regressions are ubiquitous in applied econometrics: according to (Goldsmith-Pinkham 2026), roughly half of published research in top economics and finance journals mentions “fixed effects”. Applications of fixed effects regressions can be found in almost all subfields of applied economics: Labor economists use worker and firm fixed effects to separate worker heterogeneity from firm wage premia (Abowd et al. 1999; Card et al. 2013); health economists decompose physician practice styles into physician and region fixed effects in movers designs (Molitor 2018); trade economists absorb exporter, importer, product, and time fixed effects (Head and Mayer 2014); education researchers study student-teacher panels (Chetty et al. 2014); and online marketplaces connect buyers and sellers.

The standard computational starting point for such regressions is the Frisch-Waugh-Lovell (FWL) theorem (Frisch and Waugh 1933; Lovell 1963). FWL reduces estimation to residualizing the outcome and every regressor of interest against the fixed-effect design, then running a low-dimensional regression on the residualized variables. FWL is popular because it avoids directly forming and inverting the joint Gramian of the regressors, which readily leads to out-of-memory errors for very high-dimensional fixed effects.

The workhorse method for these fixed-effect residualizations is the Method of Alternating Projections (MAP), also known as iterative demeaning or the “Zig-Zag” algorithm (Guimaraes and Portugal 2010;

Gaure 2013). Most leading software implementations of fixed-effect regression, such as `reghdfe` in Stata (Correia 2014; 2017), `fixest` (Bergé et al. 2026) in R, or `PyFixest` in Python (PyFixest Developers 2026), use MAP or MAP-based variants with acceleration.

Taking the worker-firm panel as our running example, MAP cycles through the fixed-effect dimensions: it first subtracts worker means, then firm means, then year means, and repeat until convergence. Because the worker-firm relationship is handled only through successive residual updates, never by solving a local system directly, convergence depends on how well information propagates across the graph recording which workers appear at which firms.

That co-occurrence graph is formalized by the fixed-effect Gramian (Correia 2017). Its off-diagonal blocks record co-occurrences among absorbed factors: which workers appear at which firms, which physicians practice in which regions, or which families move across which places. In a worker-firm panel, movers create paths between firms, while stayers add observations without connecting firms to one another. When this graph is sparse or poorly connected, some fixed-effect directions are nearly collinear, and MAP can require many iterations to propagate information across the graph and ultimately converge. The same graph features that determine whether worker and firm effects are identified (worker mobility, sparse mover bridges, and component structure) also determine the convergence behavior of MAP.

We propose a new preconditioner that directly encodes the co-occurrence graph. More concretely, we build a Schwarz preconditioner from local factor-pair subproblems that use the graph structure of the Gramian: for example, a worker-firm subproblem incorporates the observed links between workers and firms. A preconditioner is only useful if it approximates the inverse of the co-occurrence Gramian cheaply. We obtain such an approximation by exploiting the fact that, after a sign change, each factor-pair block is a graph Laplacian, which admits cheap approximate inverses following ideas from the Laplacian-solver literature (Spielman and Teng 2014; Gao et al. 2025). The preconditioned system is then solved iteratively via a Krylov solver.¹ In a range of benchmarks against mature implementations of the method of alternating projections, we find that on sparse, poorly connected graphs (the regime where MAP convergence deteriorates) the graph-preconditioned solver delivers substantial speedups, while on dense, well-connected graphs the preconditioner setup cost does not amortize and MAP should remain the natural default.

The rest of the paper is organized as follows. Section 2 sets up the problem of absorbing fixed effects, and Section 3 introduces the AKM model as a running example. Section 4 develops the graph structure of the fixed-effect Gramian, and Section 5 connects this structure to the convergence behavior of MAP. Sections 6 and 7 build up the factor-pair Schwarz preconditioner, starting from a general discussion of preconditioning and culminating in the construction of the novel graph-based preconditioner. Section 8 reports benchmarks on runtime, memory, and numerical equivalence; Section 9 describes the software through which the new algorithm is available; and Section 10 concludes with practical guidance.

¹The seminal Julia implementation of fixed effects regression, `FixedEffectModels.jl` (FixedEffectModels.jl Developers 2026), uses the same Krylov solver (LSMR (Fong and Saunders 2011)) as the method we propose, but only with diagonal preconditioning, which ignores the off-diagonal co-occurrence structure entirely; the contribution here is the preconditioner, not the outer iteration.

2. Absorbing Fixed Effects²

We focus on the linear model

$$y = X\beta + D\alpha + \varepsilon, \quad (1)$$

where X contains the regressors of interest, D is the fixed-effect design matrix, and α collects the fixed-effect coefficients. In high-dimensional applications, D may have hundreds of thousands or even millions of columns, so that forming the full Gramian of $Z = [X \ D]$, storing it, or applying a direct inverse is computationally infeasible. The Frisch-Waugh-Lovell (FWL) theorem provides an elegant solution to this problem.

In essence, FWL states that the coefficient on X can be obtained without explicitly estimating any element of α in the final regression. Let

$$P_D = D(D'WD)^{-1}D'W \quad (2)$$

denote the weighted projection onto the column space of D , and let $M_D = I - P_D$ denote the corresponding residualization operator. Then the coefficient of interest is

$$\hat{\beta} = (X'WM_DX)^{-1}X'WM_Dy. \quad (3)$$

Equivalently, one first residualizes y and each column of X with respect to the fixed effects, and then regresses the residualized outcome on the residualized covariates:

$$\tilde{y} = M_Dy, \quad \tilde{X} = M_DX, \quad \hat{\beta} = (\tilde{X}'W\tilde{X})^{-1}\tilde{X}'W\tilde{y}. \quad (4)$$

For a model with a single regressor of interest, this procedure corresponds to three sequential regressions: regress y on the fixed effects D and retain the residual $\tilde{y}$; regress the covariate x on the same fixed effects and retain the residual $\tilde{x}$; then regress $\tilde{y}$ on $\tilde{x}$. FWL states that the coefficient obtained from this third regression is identical to the coefficient on x in the full regression of y on x and D . With several covariates, the second step is repeated for each column of X , so that the computational burden is concentrated in repeatedly applying the fixed-effect residual operator M_D .

The remaining task is therefore to apply the same fixed-effect residualization to multiple right-hand sides. For any such right-hand side μ , whether y or one column of X , the fixed-effect fit solves

$$\hat{\alpha}_\mu = \arg \min_{\alpha} \|D\alpha - \mu\|_W^2, \quad (5)$$

where W denotes a diagonal matrix of weights. The residualized variable is then given by $\tilde{\mu} = \mu - D\hat{\alpha}_\mu$. The first-order conditions for this auxiliary least-squares problem read

$$D'W(D\hat{\alpha}_\mu - \mu) = 0, \quad (6)$$

or

$$G\hat{\alpha}_\mu = D'W\mu, \quad G = D'WD. \quad (7)$$

Equation Equation 7 is the linear system that pins down every residualization: whether of y or of one column of X , the fit reduces to a system in G with a new right-hand side. The solvers we compare divide along a basic line. MAP and its accelerated variants treat Equation 7 directly as a linear system in G ,

²Researchers employ several names for this operation: “absorbing fixed effects”, “demeaning”, “residualizing”, or applying the “within transformation”. We use these terms interchangeably throughout.

solving it by block Gauss-Seidel sweeps over its diagonal blocks. Krylov least-squares methods — LSMR and the preconditioned solver developed in this paper — instead solve the mathematically equivalent least-squares problem $\arg \min_{\alpha} \| D\alpha - \mu \|_W^2$ by iterating on the design D directly, without ever forming G ; equation Equation 7 is recovered only as its first-order condition. They differ along many further dimensions, but for our purposes the key axis of variation is the extent to which each one exploits the structure of G . The sparsity pattern and conditioning of G jointly determine how many iterations MAP requires to converge, whether a diagonal preconditioner suffices for a Krylov solver, and ultimately which solver proves fastest on a given problem. A careful analysis of the structure of G is therefore foundational for the remainder of this paper, and we turn to it in the next section.

3. A Running Example: The AKM Model

For the remainder of the paper we use the AKM model (Abowd et al. 1999) as our running example. AKM separates persistent worker heterogeneity from firm wage premia using workers who move across firms, and the regression equation of interest is

$$y_{it} = \alpha_i + \psi_{J(i,t)} + \varphi_t + x'_{it}\beta + \varepsilon_{it}, \quad (8)$$

where α_i is a worker fixed effect, $\psi_{J(i,t)}$ is the fixed effect for the firm employing worker i at time t , and φ_t is a time fixed effect.

A central observation is that the AKM specification carries a graph structure. Workers and firms form a bipartite graph whose edges are employment spells. From this we can also derive a firm-to-firm graph in which two firms are linked whenever some worker has been employed at both. Movers therefore create links between firms, while stayers add observations without linking separate firms. The year effect adds a third, low-dimensional factor that interacts with workers and firms through the same observations. Later sections demonstrate that the connectivity of this graph governs how quickly MAP can residualize.

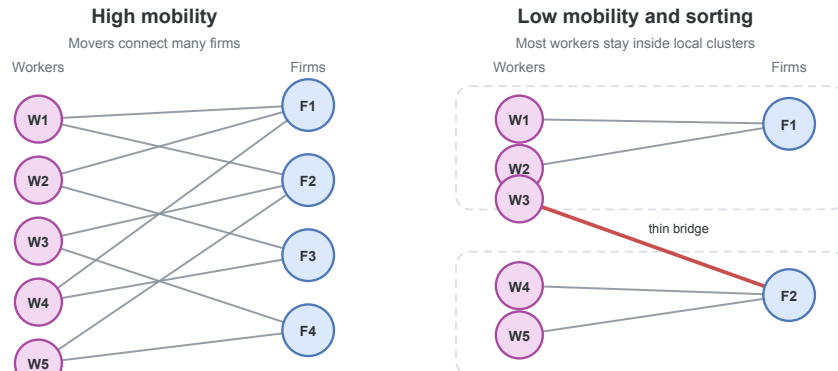


Figure 1: Worker-firm graph connectivity. High mobility creates many paths between firms. Low mobility and sorting leave the graph close to separated clusters connected by thin bridges.

The graph representation links the identification problem to the computational problem. If a worker is only ever observed at one firm, it is difficult to determine whether a high wage stems from the worker or the firm, whereas with many workers moving across many firms, each worker carries information across the graph and the two effects are easier to disentangle.

Similar graph interpretations apply to fixed-effect models beyond labor economics: physician-region panels connect movers across hospital referral regions, student-teacher panels connect students with classrooms, teachers, and schools, and trade datasets connect exporters, importers, products, and years. Online marketplaces connect buyers and sellers. In each case, the computational difficulty depends on how these fixed-effect levels are connected.

4. The Graph Structure of the Gramian

The bipartite graph introduced in Section 3 has an algebraic representation in the block structure of the Gramian $G = D'WD$ (Correia 2017). Suppose that the columns of D are ordered as worker levels, firm levels, and year levels. Then

$$G = \begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix}. \quad (9)$$

The **diagonal blocks** G_{WW} , G_{FF} , and G_{YY} contain weighted counts for workers, firms, and years. Since one observation cannot belong to two workers, two firms, or two years at the same time, these blocks are diagonal and can be solved cheaply by dividing by group counts.

The **off-diagonal blocks** are cross-tabulations: the worker-firm block C_{WF} records how often worker i is observed at firm j , and the worker-year and firm-year blocks have analogous interpretations. These blocks encode the cross-factor coupling that MAP touches only through residual updates, while a diagonal LSMR preconditioner uses only the diagonal entries of the Gramian and ignores the cross-factor blocks C_{qr} entirely.

As a small example, we construct a worker-firm panel and populate its Gramian. For simplicity, we set $W = I$.

Obs.	Worker	Firm	Year	y
1	W_1	F_1	Y_1	3.2
2	W_1	F_2	Y_2	4.1
3	W_2	F_1	Y_1	2.8
4	W_2	F_1	Y_2	3.9
5	W_3	F_2	Y_1	5.0
6	W_3	F_2	Y_2	4.5

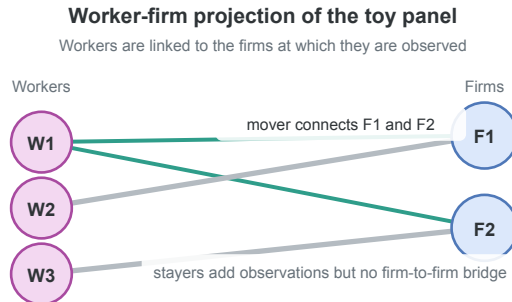


Figure 2: Worker-firm projection of the toy panel. Worker W_1 is a mover; workers W_2 and W_3 are stayers.

Worker W_1 works at both F_1 and F_2 and creates a link between the two firms; in AKM terms, W_1 is a mover. Worker W_2 stays at F_1 for two periods, and W_3 stays at F_2 for two periods. Both are stayers. The diagonal blocks are weighted count matrices. In this unweighted example, each worker is observed twice, each firm three times, and each year three times, so

$$G_{WW} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{pmatrix}, \quad G_{FF} = \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}, \quad G_{YY} = \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}. \quad (10)$$

The off-diagonal blocks are cross-tabulations between factors. The worker-firm block is

$$C_{WF} = \begin{pmatrix} 1 & 1 \\ 2 & 0 \\ 0 & 2 \end{pmatrix}. \quad (11)$$

The first row indicates that worker W_1 appears once at firm F_1 and once at firm F_2 . Workers W_2 and W_3 are stayers, appearing twice at F_1 and F_2 , respectively. The worker-year block is

$$C_{WY} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix}. \quad (12)$$

Each worker is observed once in each year. The firm-year block is

$$C_{FY} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}. \quad (13)$$

Firm F_1 appears twice in year Y_1 and once in year Y_2 ; firm F_2 has the opposite pattern. Combining the diagonal count blocks and the off-diagonal cross-tabulation blocks yields the full Gramian.

With column order $(W_1, W_2, W_3, F_1, F_2, Y_1, Y_2)$, the full unweighted Gramian is

$$G = \begin{pmatrix} 2 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 2 & 0 & 2 & 0 & 1 & 1 \\ 0 & 0 & 2 & 0 & 2 & 1 & 1 \\ 1 & 2 & 0 & 3 & 0 & 2 & 1 \\ 1 & 0 & 2 & 0 & 3 & 1 & 2 \\ 1 & 1 & 1 & 2 & 1 & 3 & 0 \\ 1 & 1 & 1 & 1 & 2 & 0 & 3 \end{pmatrix}. \quad (14)$$

The worker-firm block has a direct graph interpretation: worker W_1 is a mover who connects F_1 to F_2 , while workers W_2 and W_3 are stayers. Thus, in the worker-firm part of the graph, the information separating the two firm effects travels through W_1 . In larger panels, sparse mobility and strong sorting reproduce the same structure at scale, with cheap diagonal blocks but cross-factor blocks that determine how difficult the fixed-effect problem is.

Flipping the sign of the worker-firm cross-tabulation turns the worker-firm sub-block of G into a graph Laplacian,

$$L_{WF} = \begin{pmatrix} 2 & 0 & 0 & -1 & -1 \\ 0 & 2 & 0 & -2 & 0 \\ 0 & 0 & 2 & 0 & -2 \\ -1 & -2 & 0 & 3 & 0 \\ -1 & 0 & -2 & 0 & 3 \end{pmatrix}, \quad (15)$$

This is the worker-firm graph Laplacian: diagonal entries are weighted degrees, off-diagonal entries are negative edge weights, and the same construction works for any factor pair; Section 7 exploits this structure.³

The practical implication is that residualizing against multiple fixed effects is not, in general, a one-pass subtraction of group means. With one fixed effect, G is diagonal, so the solution is just within-group demeaning. In special balanced designs, the factor projections commute and this intuition extends to the familiar two-way double-demeaning formula (Baltagi 2021). In the AKM designs of interest, however, off-diagonal blocks such as C_{WF} are sparse and irregular: subtracting worker means changes firm means, and subtracting firm means changes worker means. The coupled system Equation 7 is therefore handled by iterative algorithms in high-dimensional applications; the next section studies the standard MAP approach.

5. Alternating Projections and Graph Connectivity

The workhorse algorithm for multi-way fixed effects is the Method of Alternating Projections (MAP), also referred to as iterative demeaning or the “zig-zag” algorithm (Guimaraes and Portugal 2010; Gaure 2013). Many packages employ MAP or its variants, frequently combined with accelerations (Berge 2018; Correia 2017); the standalone appendix describes the `fixest` acceleration strategy. Sections 3 and 4 introduced the fixed-effect graph and its algebra; this section turns to MAP itself and shows how that graph geometry governs its convergence rate.

In essence, MAP solves the FWL residualization problem by updating one fixed-effect dimension at a time. In the worker-firm-year model, worker means are first subtracted from the current residual, then firm means from the updated residual, then year means, and the procedure is iterated until convergence.

Writing the FWL normal equations Equation 7 in block form with $D = [D_W \ D_F \ D_Y]$ yields

$$\begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix} \begin{pmatrix} \alpha_W \\ \alpha_F \\ \alpha_Y \end{pmatrix} = \begin{pmatrix} D_{W'}W\mu \\ D_{F'}W\mu \\ D_{Y'}W\mu \end{pmatrix}. \quad (16)$$

Equivalently,

$$G_{WW}\alpha_W + C_{WF}\alpha_F + C_{WY}\alpha_Y = D_{W'}W\mu, \quad (17)$$

$$C_{(WF)'}\alpha_W + G_{FF}\alpha_F + C_{FY}\alpha_Y = D_{F'}W\mu, \quad (18)$$

$$C_{(WY)'}\alpha_W + C_{(FY)'}\alpha_F + G_{YY}\alpha_Y = D_{Y'}W\mu. \quad (19)$$

If the firm effects α_F and year effects α_Y were known, the first equation would deliver the worker effects α_W . Using $C_{WF} = D_{W'}WD_F$ and $C_{WY} = D_{W'}WD_Y$ to factor $D_{W'}W$ out of the right-hand side, this becomes

$$G_{WW}\alpha_W = D_{W'}W(\mu - D_F\alpha_F - D_Y\alpha_Y). \quad (20)$$

³For the worker side, $G_{WW}[i, i] = \sum_f C_{WF}[i, f]$ because each observation for worker i occurs at exactly one firm, and analogously for firms. Hence the sign-flipped block is symmetric with non-positive off-diagonal entries and zero row sums. Positive semidefiniteness follows by summing rank-one edge contributions $C_{WF}[i, f](e_i - e_f)(e_i - e_f)'$ over worker-firm edges.

The same factoring applied to the second equation, with $C_{(WF)'} = D_{F'}W D_W$ and $C_{FY} = D_{F'}W D_Y$, gives the firm update

$$G_{FF}\alpha_F = D_{F'}W(\mu - D_W\alpha_W - D_Y\alpha_Y). \quad (21)$$

The year equation is analogous.

The core insight underlying MAP is that G_{WW} , G_{FF} , and G_{YY} are diagonal matrices of weighted group counts. Consequently, each block solve, conditional on the other factors, is inexpensive: it amounts to a group-mean calculation. No general matrix inverse is required for these blocks. If C is diagonal with nonzero entries c_j , then solving $Cz = b$ reduces to elementwise division,

$$z_j = \frac{b_j}{c_j}. \quad (22)$$

Applied to G_{WW} , this divides each worker-level weighted residual sum by the worker's weighted observation count; the firm and year updates proceed analogously.

MAP exploits this fact iteratively, solving the worker block using the current firm and year effects, updating the residual, and then solving the firm and year blocks in turn. A single sweep applies this logic factor by factor. Equivalently, each update subtracts the weighted group mean of the current partial residual along the active fixed-effect dimension. In the language of numerical linear algebra, MAP is block Gauss-Seidel applied to the fixed-effect normal equations.

The cross-tabulation blocks C_{WF} , C_{WY} , and C_{FY} enter the algorithm only indirectly. For instance, the worker update is computed from the partial residual $\mu - D_F\alpha_F - D_Y\alpha_Y$, while the firm update is computed from $\mu - D_W\alpha_W - D_Y\alpha_Y$. Worker-firm, worker-year, and firm-year links are thus not solved as coupled subproblems; their effect propagates through the residual that one block update transmits to the next.

This strategy is effective when the graph is well connected. In a high-mobility worker-firm panel, many workers move across firms, so that worker and firm effects can be compared through many overlapping employment histories. A high wage at one firm can then be related to wages earned by the same workers at other firms, and a worker update rapidly alters the information seen by the next firm update, and vice versa.

When mobility is sparse, sorting is strong, or one factor is nearly nested in another, this same issue becomes simultaneously an identification problem and a numerical one. If a worker is observed only at a single firm, a high wage is difficult to attribute: it may reflect a high worker effect, a high firm effect, or both. Movers supply the links that separate these explanations. When such links are few or concentrated in narrow regions of the graph, the cross-factor information that distinguishes worker effects from firm effects is correspondingly weak. MAP can still solve the linear system, but it accesses this information only through repeated residual updates. It may therefore require many inexpensive demeaning steps before the worker and firm effects are properly separated.

This intuition admits a precise quantitative counterpart. We summarize MAP-hardness for a factor pair (q, r) through a single number, the spectral gap $1 - \rho_{qr}$. Here $\rho_{qr} = \cos^2(\theta_F)$ denotes the squared cosine of the Friedrichs angle between the q - and r -subspaces, computed from the second singular value of

the normalized cross-tabulation $H_{qr} = N_q^{-\frac{1}{2}} C_{qr} N_r^{-\frac{1}{2}}$. A gap near zero indicates that MAP contracts very slowly along the (q, r) pair (sparse mobility, near-nested structure), whereas a gap near one indicates rapid contraction. For worker-firm-year specifications the year pair is typically easy, and the difficult geometry resides in the worker-firm pair. With three or more absorbed factors this is a pairwise diagnostic on the full cyclic MAP rate rather than a formal bound; the standalone appendix provides the precise statement.

6. Graph Preconditioning for Fixed Effects

Section 5 attributed MAP’s slow convergence to thin connections in the fixed-effect graph. The same sparsity makes the Gramian G poorly conditioned, so any iterative method whose convergence is governed by the geometry of G progresses slowly. Preconditioning is the standard remedy: rather than alter the least-squares fit, one supplies the Krylov iteration with an operator $M \approx G$ so that it behaves as if applied to the better-conditioned operator

$$M^{-1}G \tag{23}$$

in place of G — for LSMR, by running the bidiagonalization in the M -weighted inner product, which clusters the eigenvalues of $M^{-1}G$. The fitted values are unchanged; only the geometry presented to the iteration is altered.

The ideal choice $M = G$ converges in a single step, but computing G^{-1} is precisely the costly operation we are trying to avoid when applying the Frisch-Waugh-Lovell theorem. A useful preconditioner must therefore approximate G^{-1} closely enough to substantially reduce the number of required iterations, while remaining cheap enough to construct and apply that the gain is not lost to setup and per-iteration overhead.

In the fixed-effect setting at hand, both requirements point to the same design principle: build M^{-1} from simpler local subproblems that preserve core geometric properties, never from the full system. In the simplest case, each subproblem involves only one fixed-effect dimension (workers, firms, or years). The corresponding block of G is diagonal, listing weighted observation counts per level, so applying M^{-1} reduces to elementwise division. This is the diagonal preconditioner used by `FixedEffectModels.jl` together with LSMR (Fong and Saunders 2011; FixedEffectModels.jl Developers 2026). A richer subproblem uses two factors jointly and incorporates the worker-firm, worker-year, and firm-year co-occurrence blocks C_{qr} that diagonal scaling discards; Section 7 develops this construction as the factor-pair Schwarz preconditioner.

6.1. Diagonal Preconditioning

Consider a labor market with a handful of very large firms alongside many small ones, such as Novo Nordisk in Danish register data or Samsung in South Korean matched employer-employee panels. In the fixed-effect normal equations, a firm with tens of thousands of worker-year observations carries a diagonal entry orders of magnitude larger than that of a five-person shop. A Krylov method applied to the unscaled system must then advance along directions of vastly different numerical scale. Rescaling the columns of D by the diagonal of $D'WD$ brings these directions onto a common footing and can substantially improve the convergence of a Krylov method such as LSMR, even though it draws no information from the worker-firm co-occurrence graph.

Diagonal scaling thus serves as an important baseline, but it uses only the count information along the diagonal of $D'WD$ and leaves the off-diagonal blocks outside the solver geometry. Bringing the cross-tabulation blocks C_{qr} into the preconditioner without forming or inverting the full Gramian requires local factor-pair problems that remain much smaller than the global system while preserving the co-occurrence structure between two fixed effects.

7. The Factor-Pair Schwarz Preconditioner

The preconditioner introduced in this section extends the diagonal baseline by incorporating selected off-diagonal blocks C_{qr} of the Gramian. These blocks record which levels of one factor co-occur with levels of another and thus encode the fixed-effect graph algebraically. In the AKM case, C_{WF} captures the worker-firm mobility structure that drives both identification and computational difficulty, information that MAP accesses only indirectly through one-factor updates.

The worker-firm panel from Section 4 makes the contrast concrete. A worker-level MAP update inverts only the diagonal count matrix $N_W = \text{diag}(2, 2, 2)$, an inverse that is cheap to compute but contains no entries linking workers to firms. The corresponding worker-firm factor-pair block augments N_W with the firm count block N_F and the cross-tabulation C_{WF} , so a local solve against this block makes direct use of the observed worker-firm edges.

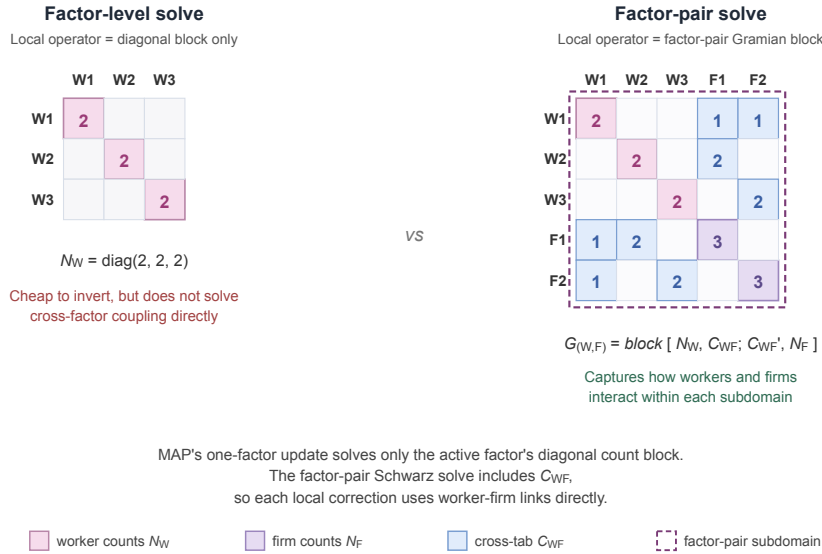


Figure 3: Local operator used by a factor-level MAP update (left) versus the factor-pair Schwarz solve (right), shown on the toy worker-firm panel of Section 4. The factor-level block is the diagonal $N_W = \text{diag}(2, 2, 2)$. The factor-pair block adds the firm count block $N_F = \text{diag}(3, 3)$ and the cross-tabulation C_{WF} in its off-diagonal positions; the dashed outline marks the worker-firm subdomain on which the local Schwarz solve operates.

Inverting these factor-pair blocks exactly would supply the right graph information, but the cost is prohibitive for large pairs. Luckily, a cheaper route is available because the factor-pair block carries a hidden Laplacian structure: flipping the sign of the cross-tabulation block C_{qr} turns it into a weighted

graph Laplacian, and the exact local inverse can be replaced by an approximate Laplacian solve based on the randomized approximate Cholesky factorizations from the Laplacian-solver literature (Spielman and Teng 2014; Gao et al. 2025).

The approximate local inverses are then assembled into an additive Schwarz preconditioner. This preconditioner is not the estimator itself but a reusable linear operator applied inside a modified, factorization-free preconditioned LSMR iteration (Fong and Saunders 2011; Arridge et al. 2014; Yang et al. 2024), so its approximations do not alter the econometric target. LSMR continues to solve the original fixed-effect least-squares fit; the preconditioner merely supplies better-conditioned search directions. Any imperfection in a local inverse is absorbed by the Krylov iteration, which terminates only once the global residual meets the requested tolerance. Once constructed, the same preconditioner is reused when residualizing the outcome and each covariate.

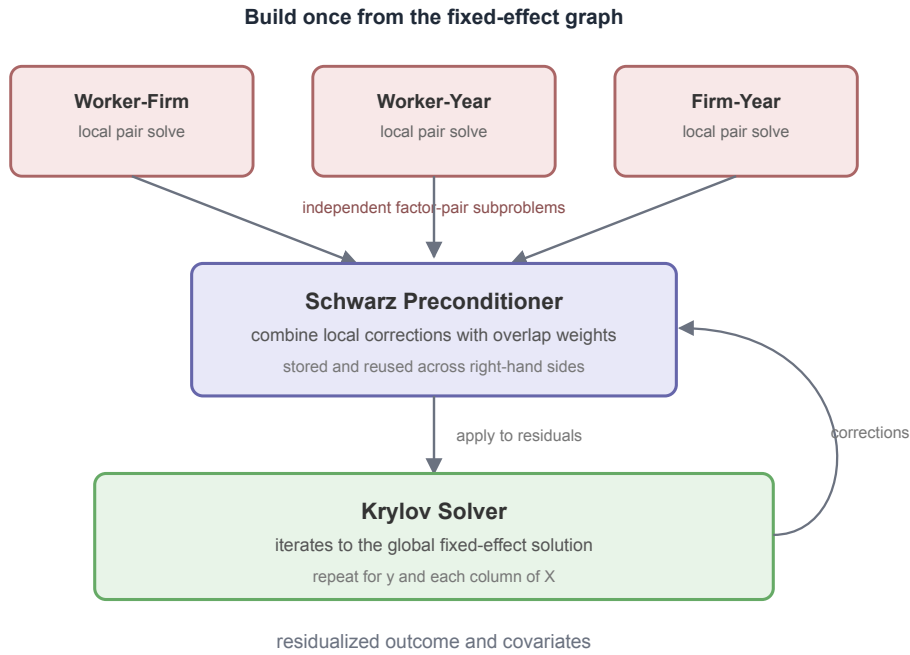


Figure 4: Macro strategy of the factor-pair preconditioner. Local pair solves are built first, combined into a reusable Schwarz preconditioner, and then applied repeatedly inside the Krylov solver for the outcome and covariates.

For a generic pair (q, r) , let $N_q = G_{qq}$ and $N_r = G_{rr}$ denote the diagonal weighted-count blocks for the two factors; these are of the same form as G_{WW} , G_{FF} , and G_{YY} in the AKM Gramian above, with the symbol N emphasizing their interpretation as count matrices. Let C_{qr} denote the cross-tabulation block. The local factor-pair block is

$$G_{qr} = \begin{pmatrix} N_q & C_{qr} \\ C_{(qr)'} & N_r \end{pmatrix}. \quad (24)$$

The preconditioner follows the additive Schwarz view of subspace correction methods (Xu 1992; Toselli and Widlund 2005). The default implementation enumerates all unordered factor pairs, builds C_{qr} for each pair, splits the induced bipartite graph into connected components, and creates one Schwarz subdomain

per component. In a worker-firm-year AKM specification this yields worker-firm, worker-year, and firm-year subdomains; usually, the worker-firm pair is the bottleneck, but the construction creates pairs for all subdomains.

For a subdomain s , let R_s be the restriction operator that extracts the fixed-effect levels of s from a global vector, and let $R_{s'}$ be its transpose, the prolongation that embeds a local vector back into the global space by padding with zeros outside s . Let A_s denote the corresponding local factor-pair operator. Because subdomains can share factor levels, the local corrections would otherwise be double-counted on those shared levels when summed; if level j appears in c_j subdomains, we therefore assign the partition-of-unity weight $\omega_j = \frac{1}{\sqrt{c_j}}$ on both restriction and prolongation, and let $\tilde{D}_s$ collect these weights. The additive Schwarz preconditioner applied to a residual r is then

$$M^{-1}r = \sum_{s=1}^{N_s} R_{s'} \tilde{D}_s A_s^+ \tilde{D}_s R_s r, \quad (25)$$

where the sum runs over the N_s Schwarz subdomains and A_s^+ denotes an approximate local solve. Reading the formula from right to left, $R_s r$ restricts the global residual to subdomain s , $\tilde{D}_s$ applies the partition weights, A_s^+ solves the local system approximately, and $R_{s'}$ prolongates the weighted correction back to the global space. Using identical weights on restriction and prolongation makes the additive preconditioner symmetric positive definite, which is the property the preconditioned LSMR iteration requires of M .

Algorithm 1. Factor-Pair Schwarz Preconditioner

Inputs

- Observation-level factor codes for Q absorbed dimensions.
- Diagonal weights W and a local solver configuration.
- Krylov residual r in coefficient space.

Preconditioner setup

- Enumerate all unordered factor pairs (q, r) with $q < r$.
- For each pair, build weighted count blocks N_q, N_r , and the weighted cross-tabulation C_{qr} .
- Split the induced bipartite graph into connected components and create one Schwarz subdomain s per component.
- If fixed-effect level j appears in c_j subdomains, store the partition weight $\omega_j = \frac{1}{\sqrt{c_j}}$.
- For each subdomain, sign-flip one side to obtain a local Laplacian, project the local right-hand side off the component constant, and build a zero-mean pseudoinverse solve.
- Use a Schur-complement local solver: small reduced systems are solved directly, while larger reduced SDD/Laplacian systems are solved with randomized approximate Cholesky.

Krylov application

- Initialize $z = 0$.
- For each subdomain s , form $h_s = \tilde{D}_s R_s r$.
- Compute the approximate local correction $u_s \approx A_s^+ h_s$ on the normalized subspace.
- Accumulate $z \leftarrow z + R_{s'} \tilde{D}_s u_s$.
- Return $z = M^{-1}r$.

8. Benchmarks

The preceding sections yield a computational prediction: graph preconditioning should not outperform MAP universally, but it should help precisely when the fixed-effect graph causes factor-by-factor demeaning via MAP to propagate information slowly. The benchmarks test this prediction across settings that range from controlled synthetic stress tests to standard public benchmark datasets.

The benchmarks address three practical questions. First, do the runtime gains appear in the regression interfaces that practitioners actually use? The algorithmic difference resides in the fixed-effect absorption step, but benchmarking only a demeaning kernel would omit software costs that matter in practice: parsing the model, constructing fixed-effect encodings, preparing right-hand sides, reusing solver state across variables, and returning the coefficient estimate. Second, is the memory overhead of storing factor-pair information small enough to keep the method viable on commodity hardware? Third, does the new solver return the same least-squares solution as MAP up to numerical tolerance, despite the implementations using different convergence checks and stopping thresholds?

For this reason, and to maintain comparability with the public PyFixest benchmark suite, the runtime benchmarks use the package-level regression APIs. Each runtime includes model setup, construction of the fixed-effect representation, residualization of the outcome and covariates, and estimation of the coefficient of interest. The results should be interpreted as software-level timings for the same econometric problem, not as isolated timings of the demeaning routine alone.

To connect the timings back to the graph-theoretic mechanism, the runtime tables also report a compact hardness heuristic. For the relevant factor pair, we compute $\rho_{qr} = \sigma_2(H_{qr})^2$ after removing the component-wise constant singular direction, and we report the gap $1 - \rho_{qr}$ together with the observation share of the component attaining the reported value. Smaller gaps indicate harder two-factor MAP geometry; a large component share indicates that this hard geometry is not confined to a negligible portion of the sample. With three or more fixed effects, these serve as pairwise diagnostics, not full convergence bounds. Our benchmarks are organized in three stages. First, we use controlled AKM synthetic datasets that mimic worker-firm mobility and sorting patterns to isolate the mechanism underlying the theory: MAP slows when connectivity weakens, while a factor-pair preconditioned Krylov method remains stable. Second, we turn to standard synthetic benchmarks: the simple/difficult DGPs from the fixest benchmark suite and the synthetic datasets currently collected by Sergio Correia. Third, we use empirical datasets from the same Correia collection, because synthetic DGPs can miss the irregularities of real networks.

We compare the new preconditioned implementation to three existing solver strategies: a vanilla MAP backend, a mature accelerated MAP implementation, and a diagonally-preconditioned Krylov solver. The benchmarked backends are:

Backend	Package	Algorithm
rust-map	PyFixest	Vanilla Rust MAP backend without acceleration.
fixest	R fixest	Sophisticated MAP implementation with Irons-Tuck acceleration, coefficient-space routines, and additional heuristics (Bergé et al. 2026); see the standalone appendix.

Backend	Package	Algorithm
FEM.jl	FixedEffectModels.jl	LSMR, a Krylov method with diagonal preconditioning (Fong and Saunders 2011; FixedEffectModels.jl Developers 2026).
within	PyFixest	The same LSMR Krylov method as FEM.jl, but with factor-pair Schwarz preconditioning in place of diagonal scaling.

All reported CPU times are medians across three benchmark iterations run on an Apple M4 Mac mini with 10 CPU cores and 16 GB of memory running macOS 15.3.1.

We do not include `reghdfe` directly in the benchmark tables because Stata is not open source and we lack a license. `reghdfe` is a mature accelerated-MAP implementation with a conjugate-gradient option, single-ton-observation pruning, and accelerated demeaning (Correia 2014; 2017); algorithmically, it belongs to the same family as `fixest`’s accelerated MAP.

8.1. Runtime Benchmarks

8.1.1. Controlled Synthetic Benchmarks: AKM Mobility and Sorting

We begin with controlled experiments on synthetic datasets that reproduce the salient features of AKM panels. Each experiment varies a single graph feature while holding the remainder of the data-generating process fixed, so that any change in runtime can be attributed to graph geometry.

The first experiment progressively reduces worker mobility. Theory predicts *ex ante* that MAP should slow down as mobility declines, since one-factor demeaning has fewer worker moves through which to propagate information across firms. The preconditioned Krylov algorithm should therefore become relatively more competitive in the low-mobility designs, where its factor-pair preconditioner can exploit the worker-firm graph directly.

Mobility benchmark ($n = 1M$).

Scenario	Gap (share)	rust-map	fixest	FEM.jl	within
akm_mobility_1	1.38×10^{-1} (1.00)	0.29s	0.16s	0.28s	0.51s
akm_mobility_2	3.41×10^{-2} (1.00)	1.10s	0.36s	0.58s	0.38s
akm_mobility_3	5.18×10^{-3} (0.61)	12.48s	1.31s	1.80s	0.33s
akm_mobility_4	3.04×10^{-4} (0.27)	51.95s	3.42s	2.78s	0.34s
akm_mobility_5	1.65×10^{-3} (0.53)	63.23s	4.17s	3.34s	0.35s
akm_mobility_6	7.57×10^{-4} (0.37)	–	5.27s	3.86s	0.35s

Note: AKM-style panel with 1M observations, one covariate, and worker, firm, and year fixed effects. Moving down the table reduces worker mobility within a 10-period panel, thereby thinning the worker-firm graph. Lower mobility renders the worker-firm pair more difficult for MAP and correspondingly more favorable to the preconditioned method. Gap is defined as $1 - \rho_{WF}$, with $\rho_{WF} = \cos^2(\theta_F)$ for the worker-firm pair; parentheses report the observation share of the component attaining the gap. Hardness diagnostics are computed on representative 100K draws using the same DGP parameters.

The mobility benchmark conforms to the predicted pattern. When mobility is high, preconditioning yields little advantage: `fixest` is the fastest backend in `akm_mobility_1`, and `within` incurs setup costs that are not yet offset by improved convergence. As mobility declines, the worker-firm gap shrinks from 1.38×10^{-1} to below 10^{-3} , and MAP runtimes rise sharply. `within` remains between 0.33s and 0.51s across all designs. In the lowest-mobility configurations, the preconditioned method is the fastest backend because

it operates on the worker-firm graph directly rather than propagating information through many one-factor sweeps.

The second experiment progressively increases the sorting of workers to firms. Stronger sorting drives the worker-firm graph closer to block diagonal form, since movers tend to connect firms within similar groups rather than across distant regions of the graph. The same theoretical argument predicts that MAP should again lose ground as sorting intensifies, while the preconditioned method should prove less sensitive because its local factor-pair solves capture this cross-factor structure.

Sorting benchmark ($n = 1\text{M}$).

Scenario	Gap (share)	rust-map	fixest	FEM.jl	within
akm_sorting_1	7.62×10^{-3} (0.83)	6.79s	0.81s	1.43s	0.32s
akm_sorting_2	2.41×10^{-3} (0.62)	9.73s	1.12s	1.68s	0.34s
akm_sorting_3	1.50×10^{-4} (0.59)	10.31s	1.24s	1.67s	0.33s
akm_sorting_4	2.14×10^{-5} (0.51)	14.94s	1.58s	1.87s	0.33s
akm_sorting_5	3.97×10^{-5} (0.53)	25.77s	1.80s	1.94s	0.34s

Note: AKM-style panel with 1M observations, one covariate, and worker, firm, and year fixed effects. Moving down the table raises the degree of sorting among movers, pushing the worker-firm graph closer to block diagonal form. Stronger sorting weakens cross-block information flow and thereby raises the value of factor-pair preconditioning. Gap is $1 - \rho_{WF}$ for the worker-firm pair; parentheses report the observation share of the component attaining the gap. Hardness diagnostics are computed on representative 100K draws using the same DGP parameters.

The sorting benchmark illustrates the same mechanism from a complementary direction. As movers become more sorted across firms, the graph approaches a set of separated blocks, the worker-firm gap falls by two orders of magnitude, and MAP-based runtimes climb correspondingly. `within` remains essentially flat, ranging from 0.32s to 0.34s, because the factor-pair preconditioner already encodes the worker-firm coupling that sorting renders difficult for one-factor residual updates to discover.

8.1.2. Standard Synthetic Benchmarks: fixest DGPs + Correia Synthetic

The AKM benchmarks above were designed to isolate a single mechanism. We now turn to synthetic benchmark datasets that have become standard reference cases in fixed-effect software: they are public, easily reproducible, and already in use for comparing implementations.

The first family is the simple-versus-difficult benchmark data generating process from `fixest` (Bergé et al. 2026), which is also used in the PyFixest benchmark suite (PyFixest Developers 2026). Both designs employ 10M observations, one covariate, and three fixed effects (worker, firm, year). The simple design features dense random mobility, whereas the difficult design features a sparse, nearly nested worker-firm structure. We would therefore expect the simple design to be genuinely “simple” for MAP, since one-factor demeaning can propagate information rapidly through a well-connected graph. Conversely, the difficult design should be challenging for MAP because the worker and firm effects are nearly collinear. Graph preconditioning should perform well on the difficult design, but may fail to amortize its setup cost on the “simple” design.

Simple vs. difficult design (10M observations, 3 FE).

Design	Gap (share)	rust-map	fixest	FEM.jl	within	torch-cuda
simple (dense graph)	8.57×10^{-1} (1.00)	2.01s	0.93s	2.27s	10.53s	4.73s

Design	Gap (share)	rust-map	fixest	FEM.jl	within	torch-cuda
difficult (sparse graph)	1.67×10^{-5} (1.00)	382.9s	32.7s	28.7s	3.25s	8.73s

Note: Medians over three full regression calls. Both designs use 10M observations, one covariate, and three fixed effects. Gap denotes $1 - \rho_{WF}$ for the worker-firm pair, reported from the generated 1M version of the same DGP family; the runtime rows use the identical simple/difficult graph construction at 10M observations. torch-cuda denotes the PyFixest GPU LSMR backend with diagonal preconditioning (the same algorithm as FixedEffectModels.jl), run on an NVIDIA CUDA device; the local benchmark machine does not possess a CUDA GPU. The standalone within demeaning API decomposes one-shot runtime into reusable solver construction and batch solve: simple design 5.76s + 1.51s (roughly 75% setup); difficult design 0.40s + 1.00s (roughly 29% setup). PyFixest regression overhead is incurred on top of these figures.

The fixest DGPs exhibit the same tradeoff in its cleanest form. On the simple design, where the graph is dense and the worker-firm gap is large, fixest with Irons-Tuck acceleration is fastest at 0.93s while within is slowest at 10.53s; the preconditioner setup cost is not recouped: MAP already converges rapidly. As the caption indicates, roughly three quarters of within’s one-shot demeaning time on the simple design consists of reusable solver setup rather than the batch solve itself.

On the difficult design, the ranking reverses. MAP convergence degrades sharply: rust-map without acceleration fails to finish within six minutes, and fixest requires 32.7s, whereas within completes in 3.25s because its factor-pair preconditioner captures the sparse worker-firm coupling directly. The diagnostic gap is nearly zero, so the table measures the regime in which MAP requires many sweeps to disentangle worker and firm effects. The setup share of the standalone demeaning time correspondingly falls to about 29%. The difficult design is precisely the setting in which pair information pays off, while the simple design is sufficiently easy that constructing the graph preconditioner constitutes a cost rather than a shortcut.

The GPU backend (torch-cuda) runs the same diagonally preconditioned LSMR algorithm as FixedEffectModels.jl, but on an NVIDIA CUDA device. On the simple design it is not competitive with CPU fixest (4.73s versus 0.93s), since host-to-device transfer and kernel launch overheads outweigh the gain from parallelizing already inexpensive iterations. On the difficult design, GPU parallelism reduces runtime to 8.73s (roughly three times faster than CPU FEM.jl), but the iteration count remains governed by the quality of the diagonal preconditioner, which is limited on this graph. within at 3.25s remains faster on CPU by employing a factor-pair preconditioner that captures the cross-factor coupling the diagonal cannot. Hardware acceleration and stronger preconditioning address different bottlenecks; on poorly conditioned designs, the preconditioner is the larger lever.

The second family of benchmarks comprises synthetic datasets drawn from the Correia HDfE benchmark collection. These datasets span a broader set of graph shapes: complete bipartite matching, uniform random matching with varying degrees of connectivity, assortative matching, and a small path-like design. They provide an independent public reference set for comparing the same software backends outside the AKM generator.

Correia synthetic benchmarks.

Dataset	Gap (share)	rust-map	fixest	FEM.jl	within
synthetic-complete	1.000 (1.00)	0.074s	0.062s	0.044s	0.114s
synthetic-uniform-easy	0.651 (1.00)	0.122s	0.079s	0.083s	0.117s

Dataset	Gap (share)	rust-map	fixest	FEM.jl	within
synthetic-uniform-hard	0.184 (1.00)	0.367s	0.251s	0.353s	0.957s
synthetic-uniform-harder	0.0249 (1.00)	0.994s	0.952s	0.575s	0.514s
synthetic-assortative	0.00133 (0.70)	26.22s	3.02s	2.99s	1.46s

Note: Medians over three runs. Gap denotes $1 - \rho$ for the id1-id2 pair after the same singleton pruning; parentheses report the observation share of the component attaining the gap. Smaller gaps correspond to slower two-way MAP geometry. The synthetic-zigzag dataset is omitted because every backend terminates with a numerical error rather than a converged solution under default settings, making it a stress case rather than a runtime comparison.

These results are less clear than the controlled AKM experiments, which is itself informative. Several datasets are small enough, or sufficiently well connected, that setup cost matters as much as conditioning; on the complete and easier uniform designs, the gap is large and low-overhead methods perform well. The harder rows tell a different story. By synthetic-uniform-harder, the gap has fallen to 2.49×10^{-2} and within is already the fastest backend. On the assortative benchmark, the preconditioned method exhibits its clearest advantage: sorting generates cross-factor structure that one-factor updates only handle slowly.

8.1.3. Standard Real-Data Benchmarks: Correia Collection

We next turn to real benchmark data from the Correia collection. Synthetic data sets match the overall shape of empirical co-occurrence graphs but smooth away the messy details that often drive runtime: a few units that show up far more often than the others, thin connections between otherwise dense groups, lots of small disconnected pieces, and interactions between identifiers that go beyond a clean two-way pair. Real data carries these features by default, and therefore tests the solvers in conditions that controlled DGPs only approximate.

Correia real-data benchmarks.

Dataset	Gap (share)	rust-map	fixest	FEM.jl	within
credit	0.402 (1.00)	0.177s	0.134s	0.158s	0.139s
soccer	0.889 (1.00)	0.018s	0.014s	0.016s	0.031s
enron	0.00704 (0.98)	4.37s	0.749s	0.689s	0.537s
github	0.000630 (0.32)	86.44s	4.50s	4.60s	0.380s
patents	0.000518 (0.95)	19.66s	1.29s	1.43s	0.826s
workers	0.000274 (0.63)	128.34s	5.55s	4.94s	0.491s
schools	0.00221 (1.00)	11.04s	1.10s	1.28s	0.224s
directors	0.000512 (0.30)	4.26s	0.216s	1.21s	0.300s

Note: Medians over three runs from the `singleton_drop = true` rows of `pyfixest/benchmarks/results/correia-benchmarks.csv`. The gap is $1 - \rho$ for the id1-id2 pair after the same singleton pruning; parentheses report the observation share of the component attaining the gap. A small gap with a limited component share, as in `directors`, indicates a hard subcomponent but not necessarily whole-sample MAP difficulty.

The empirical datasets present the same lesson in less stylized form. Accelerated MAP is difficult to outperform on small or compact graphs such as `credit` and `soccer`, where the gaps are large. The `directors` row illustrates why the component share is useful: the worst component is hard, but it contains about thirty percent of the observations, so the full problem is not as costly for MAP as the gap alone would suggest. On larger networks with hard components covering a substantial portion of the sample, particularly `enron`, `github`, `patents`, `workers`, and `schools`, the factor-pair preconditioner is fastest by a wide margin.

8.2. Poisson / PPML Benchmark

Generalized linear models with high-dimensional fixed effects are typically estimated by iteratively reweighted least squares (IRLS). Each IRLS step fits a weighted least squares problem in which the response and covariates are demeaned against the fixed effects, with weights that are updated between iterations (Correia et al. 2020; Stammann 2018). The demeaning operation is identical to the process described in the prior sections. Any acceleration of fixed-effect demeaning therefore propagates into the GLM runtime, multiplied by the number of IRLS iterations.

The IRLS structure also enlarges the window over which a preconditioner setup cost can amortize. A factor-pair preconditioner depends on the fixed-effect structure, which is fixed across iterations, and on the IRLS weights, which change. If the IRLS weights do not change too much, a “slightly stale” preconditioner will still be effective. We exploit this by constructing the preconditioner once and then reusing the stale” preconditioner on subsequent IRLS iterations. Staleness slows the outer Krylov solver but does not bias its solution; the iteration still converges to the correct demeaned residuals. The preconditioner is refreshed only when the Krylov solver exceeds a chosen iteration threshold, so the construction cost is paid a small number of times per regression rather than once per IRLS step.

We benchmark this strategy on the simple-versus-difficult DGPs from the `fixest` benchmark suite (Bergé et al. 2026) at $n = 1\text{M}$ observations, $k = 10$ covariates, and two or three fixed effects (worker-year, or worker-firm-year), using the same iteration protocol as the OLS benchmarks. The compared backends are R `fixest`’s `fepois`, `GLFixedEffectModels.jl`, and two PyFixest `fepois` backends: the default unpreconditioned `rust-map` and the preconditioned `within` solver.

Poisson benchmarks (1M observations, k=10 covariates).

Design	FE	<code>fixest</code>	<code>rust-map</code>	<code>GLFEM.jl</code>	<code>within</code>
simple (dense graph)	2	1.92s	3.06s	8.83s	6.21s
difficult (sparse graph)	2	1.91s	3.05s	7.93s	6.40s
simple (dense graph)	3	7.80s	25.29s	~ 48s	13.33s
difficult (sparse graph)	3	309.2s	failed	~ 800s	8.89s

Note: Medians over three full IRLS regression calls at $n = 1\text{M}$ and $k = 10$ covariates. `fixest` is R `fixest::fepois`; `rust-map` and `within` are the PyFixest `fepois` routine with the unpreconditioned MAP backend and the factor-pair preconditioned solver, respectively; `GLFEM.jl` is `GLFixedEffectModels.jl`. The two `GLFEM.jl` entries marked ~ are read approximately from a follow-up run because the harness CSV for those cells contained a subprocess error rather than a recorded time. “failed” indicates that all three iterations of `rust-map` reached the 10000-iteration MAP cap without converging.

The patterns observed in the OLS benchmarks carry over. With two fixed effects, all four backends complete in comparable time on both designs and R `fixest` is fastest: the worker-year pair alone does not create a regime in which the preconditioner amortizes its setup cost, even with the IRLS-induced reuse. Adding the firm fixed effect changes the picture. On the three-FE simple design MAP still converges and `fixest` remains fastest, but the unaccelerated `rust-map` slows to 25s, `GLFEM.jl` to roughly 48s, and `within` lands between them at 13s. The three-FE difficult design provides the sharpest contrast: `rust-map` does not converge within the iteration cap, `GLFEM.jl` takes on the order of 800s, `fixest`’s IRLS loop takes several minutes with large run-to-run variance, and `within` finishes in under nine seconds, roughly 35 times faster than `fixest` and two orders of magnitude faster than `GLFEM.jl`. The factor-pair preconditioner

captures the same sparse worker-firm coupling that drove the OLS benchmark results, and the IRLS outer loop inherits the gain without modification.

8.3. Memory Use

MAP is cheap on memory: a sweep only needs the current residuals and per-level group sums. A graph preconditioner, by contrast, has to keep factor-pair structure around between iterations. A practically relevant question is therefore how much more memory the preconditioned Krylov solver requires relative to MAP.

We measure peak resident set size (peak RSS), the largest quantity of physical memory consumed by the process during a run, on the simple and difficult fixed-effect DGPs. These serve as representative probes rather than special memory cases: the dominant storage terms are the data matrix, the fixed-effect encodings, and the reusable factor-pair objects, so the results should largely generalize across datasets of comparable dimensions. We do not employ this section to compare against `fixest` or `FixedEffectModels.jl`, because cross-language peak RSS also reflects R and Julia runtime overhead, data-loading choices, garbage collection, and package internals. The diagnostic of interest is the incremental memory cost of substituting the preconditioned Rust backend for Rust MAP within the same Python package. Because the surrounding regression code is shared, this comparison isolates the difference attributable to the demeaning strategy. Both backends are executed in isolated processes and report peak RSS via `ru_maxrss`.

Memory footprint (3 FE, $k = 10$).

Design	Gap (share)	rust-map	within
<i>100K observations</i>			
simple (dense graph)	0.857 (1.00)	428 MB	487 MB
difficult (sparse graph)	0.00130 (1.00)	432 MB	479 MB
<i>1M observations</i>			
simple (dense graph)	0.857 (1.00)	1,188 MB	1,703 MB
difficult (sparse graph)	1.67×10^{-5} (1.00)	1,263 MB	1,398 MB

Note: Peak RSS denotes peak resident set size, measured from isolated Python processes. 10 covariates, three fixed effects. These two DGPs serve as representative probes; we do not claim that memory behavior is design-specific. Gap denotes $1 - \rho_{WF}$ for the worker-firm pair in the corresponding generated design.

At 100K observations, the preconditioner adds roughly 50 MB on both the easy and the hard graph. At 1M observations the overhead is larger in absolute terms (135–515 MB), but it remains modest relative to the data footprint of a panel with 10 covariates. This is the expected tradeoff: the preconditioned solver consumes more memory than MAP, yet the additional storage for factor-pair co-occurrences, partition weights, and local approximate Cholesky factors remains relatively lightweight in comparison to the memory requirements of the full regression problem.

8.4. Numerical Equivalence

When introducing a new fixed-effect solver, the burden falls on the implementation to demonstrate that it matches existing solutions. We should not take exact agreement for granted: MAP and LSMR-style routines use different convergence checks, residual norms, stopping thresholds, and iteration caps. The tables below report targeted diagnostics for the PyFixest Rust backends used in this paper and for coefficient agreement with external software.

We use the 100K-observation simple and difficult data generating processes as diagnostic cases. The simple design examines the easy regime where both methods should agree almost exactly. The difficult design examines the regime where conditioning is poor and small differences in stopping rules are most likely to be amplified. The graph diagnostic distinguishes these cases sharply: the worker-firm gap is approximately 0.857 in the simple design and approximately 0.00130 in the difficult design. The table below collects slope coefficients and cross-software differences for all four backends. Additional within-PyFixest residual and fixed-effect diagnostics, a tolerance-scaling experiment, and the exact convergence rules used by each backend are reported in the standalone appendix. The table should therefore be interpreted as a default-settings agreement check, not as a common-tolerance accuracy comparison.

Coefficient agreement (100K observations, 3 FE, $k = 10$).

Design	Backend	$\hat{\beta}_1$	Avg diff	Max diff
simple	rust-map	0.99914206	–	–
	within	0.99914206	1.5×10^{-14}	3.5×10^{-14}
	fixest	0.99909173	2.2×10^{-5}	5.0×10^{-5}
	FEM.jl	0.99914206	2.1×10^{-12}	5.3×10^{-12}
difficult	rust-map	1.00086122	–	–
	within	1.00086098	1.4×10^{-7}	3.4×10^{-7}
	fixest	1.00081490	2.0×10^{-5}	5.0×10^{-5}
	FEM.jl	1.00086098	1.4×10^{-7}	3.4×10^{-7}

Note: $\hat{\beta}_1$ is the slope coefficient on x_1 . Differences are absolute slope-coefficient deviations from rust-map, averaged (Avg) and maximized (Max) across all 10 covariates. within is the PyFixest preconditioned Rust backend. fixest and FixedEffectModels.jl are invoked through their own regression APIs. The simple and difficult designs have worker-firm gaps of 0.857 and 0.00130, respectively, so the table compares numerical agreement in both an easy and a hard graph regime. The largest cross-backend gap occurs at the fifth decimal, which is below the precision at which applied coefficients are typically reported.

The within and FEM.jl rows are almost identical to rust-map at the reported defaults. R fixest is also close, but not at machine precision: the largest coefficient gap is approximately 5×10^{-5} . This is unsurprising because the packages do not share a stopping rule.

9. Software Availability

The solver studied in this paper is available as open-source software through the within project (within Developers 2026a). The computational core is implemented in Rust and exposed through Rust, Python, and R interfaces; each language binding invokes the same underlying solver. This matters for reproducibility and adoption: the method can be used directly from Rust, called from Python workflows, or invoked from R without reimplementing the algorithm.

Interface	Package	Registry	Install
Rust	within	crates.io (within Developers 2026b)	cargo add within
Python	within-py	PyPI (within Developers 2026c)	pip install within-py
R	withinr	py-econometrics R-universe (within Developers 2026d)	install.packages("withinr", repos = ...)

The Rust crate exposes the lower-level solver together with its configuration types. The Python and R packages provide solver-level `solve` and `solve_batch` APIs for residualizing one or several right-hand sides. The Python package is imported as `within` and is also integrated into PyFixest (PyFixest Developers 2026).

10. Conclusion

High-dimensional fixed effects are typically presented as an econometric device for absorbing many categorical controls while estimating a lower-dimensional coefficient of interest. Computationally, the same specification defines a graph, and the connectivity of that graph affects how quickly information propagates between factors during residualization.

MAP remains the right default when the graph is dense and well connected. Its iterations are inexpensive, mature implementations incorporate strong accelerations, and a graph preconditioner may spend more time building structure than it saves. This is precisely what occurs in the “simple” fixest data generating process and in the smaller, more compact Correia benchmark data sets.

The preconditioned solver is attractive when the design contains a sparse cross-factor pair of fixed effects. In AKM applications this is typically the worker-firm pair: low mobility, strong sorting, thin bridges between firm groups, or near-nesting all cause one-factor demeaning to propagate information slowly across the graph. Similar patterns arise in physician-patient, student-teacher, exporter-importer, and product-market designs whenever the identifying moves are sparse or concentrated within clusters.

What matters is therefore graph connectivity, not sample size alone. If MAP converges in a few sweeps, there is little to improve. If the fixed-effect pair has many small components, weak bridges, mostly within-cluster movers, or a nearly nested structure, a factor-pair preconditioner provides the Krylov iteration with information that MAP recovers only after many residual updates. The preconditioner setup is also amortizable: the same construction can be reused across multiple demeaning calls on a fixed fixed-effect structure, which makes the approach particularly attractive for GLM fitting with fixed effects, where IRLS issues many demeaning solves per regression.

In practice, runtime itself is the simplest signal to decide if one should probe alternative solvers to MAP. In our experience, if MAP takes more than ten seconds on a routine fit, the fixed-effect structure may be putting the problem in a hard regime where one-factor demeaning propagates information slowly across the graph, and trying the preconditioned solver is typically cheaper than diagnosing the geometry by hand. Concrete heuristics to automatically select MAP vs a preconditioned Krylov solver are work in progress.

Appendix: Local Factor-Pair Solver Details

This appendix presents the local-solver machinery summarized in Section 7. The local factor-pair solve succeeds because each pair block possesses a hidden Laplacian structure. For a pair (q, r) ,

$$G_{qr} = \begin{pmatrix} N_q & C \\ C' & N_r \end{pmatrix}. \quad (26)$$

After flipping the sign of one side, this expression becomes

$$L_{qr} = \begin{pmatrix} N_q & -C \\ -C' & N_r \end{pmatrix}. \quad (27)$$

L_{qr} constitutes a valid weighted graph Laplacian: it is symmetric, its off-diagonal entries are non-positive, and its row sums vanish. The row-sum property holds because each observation contributes exactly one level on each side, so that the diagonal count for level j of factor q equals the sum of its cross-tabulation row, $N_q[j, j] = \sum_k C_{qr}[j, k]$ (and symmetrically for r).

The sign flip can be expressed as a similarity transform. With $T = \begin{pmatrix} I & 0 \\ 0 & -I \end{pmatrix}$ one verifies that $L_{qr} = TG_{qr}T$, and since $T^2 = I$, the system $G_{qr}z = h$ is equivalent to $L_{qr}u = Th$ with $z = Tu$. Because the Laplacian L_{qr} is singular on each connected component, the implementation adopts the zero-mean component solution: the local right-hand side is projected off the constant direction, and the returned correction has zero mean on that component. The fitted correction is invariant to this normalization. The approximate Schur complement and approximate Cholesky steps, introduced below, can produce small row-sum deficits that break the strict Laplacian property; when this occurs the implementation applies *Gremban augmentation* (Gremban 1996), which appends a single grounded node connected to all others in order to absorb the deficit and restore a valid Laplacian on the augmented system.

The default local solver eliminates the larger side of the bipartite graph, so that the reduced system is formed on the smaller factor. If the eliminated side is q , the Schur complement takes the form

$$S = N_r - C' N_q^{-1} C. \quad (28)$$

(The same Schur complement is obtained whether elimination is carried out on G_{qr} or on the sign-flipped Laplacian L_{qr} , since $(-C')(-C) = C'C$.)

Exact elimination can generate dense fill. In worker-firm language, eliminating a worker who visited d firms produces a clique among those firms with $\binom{d}{2}$ edges. For large factor pairs, within therefore replaces this exact elimination with an *approximate* Schur step: each eliminated vertex's clique is replaced by a randomly sampled spanning tree on its neighbours, with $d - 1$ edges in place of $\binom{d}{2}$, and tree weights chosen so that the expected reduced Laplacian equals the exact Schur complement (an unbiased estimator) (Gao et al. 2025). The reduced system is then solved either by a dense factorization when small, or by randomized approximate Cholesky on larger reduced SDD/Laplacian systems (Spielman and Teng 2014; Gao et al. 2025), which applies the same clique-tree sampling idea sequentially as it eliminates the remaining vertices. The local factors remain sparse and inexpensive to apply, while the outer Krylov iteration corrects the approximation error globally.

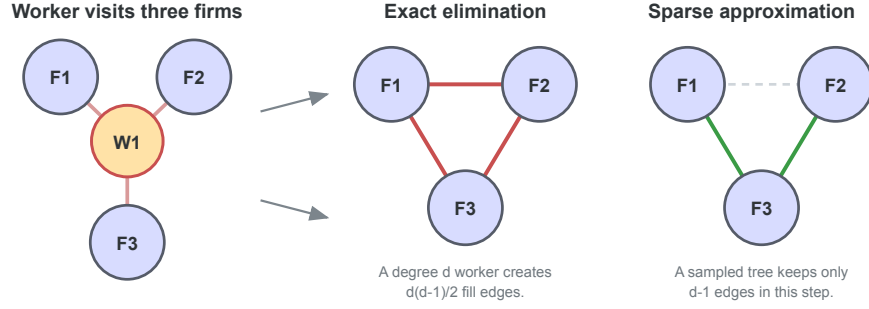


Figure 5: Approximate Schur step. Exact elimination of a vertex with d neighbours yields a clique with $\binom{d}{2}$ fill edges (left); the approximate variant replaces this clique with a randomly sampled spanning tree of $d - 1$ edges (right), where the weights are chosen so that the expected reduced Laplacian equals the exact Schur complement.

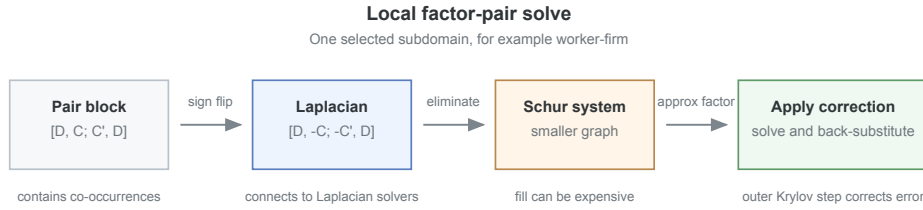


Figure 6: Local solve pipeline: sign flip, Schur reduction, approximate factorization, and back-substitution.

References

- Abowd, John M., Francis Kramarz, and David N. Margolis. 1999. “High Wage Workers and High Wage Firms.” *Econometrica* 67 (2): 251–333. <https://doi.org/10.1111/1468-0262.00020>.
- Arridge, Simon R., Marta M. Betcke, and Lauri Harhanen. 2014. “Iterated Preconditioned LSQR Method for Inverse Problems on Unstructured Grids.” *Inverse Problems* 30 (7): 75009. <https://doi.org/10.1088/0266-5611/30/7/075009>.
- Baltagi, Badi H. 2021. *Econometric Analysis of Panel Data*. 6th ed. Springer.
- Berge, Laurent. 2018. *Efficient Estimation of Maximum Likelihood Models with Multiple Fixed-Effects: The R Package Fenmlm*. Technical Report Nos. 2018–13.
- Bergé, Laurent R., Kyle Butts, and Grant McDermott. 2026. “Fixest: A Fast and Feature-Rich Framework for Econometric Estimations in R.” <https://doi.org/10.48550/arXiv.2601.21749>.
- Card, David, Jörg Heining, and Patrick Kline. 2013. “Workplace Heterogeneity and the Rise of West German Wage Inequality.” *The Quarterly Journal of Economics* 128 (3): 967–1015. <https://doi.org/10.1093/qje/qjt006>.
- Chetty, Raj, John N. Friedman, and Jonah E. Rockoff. 2014. “Measuring the Impacts of Teachers II: Teacher Value-Added and Student Outcomes in Adulthood.” *American Economic Review* 104 (9): 2633–79. <https://doi.org/10.1257/aer.104.9.2633>.
- Correia, Sergio. 2014. “Reghdfe: Stata Module to Perform Linear or Instrumental-Variable Regression Absorbing Any Number of High-Dimensional Fixed Effects.” <https://github.com/sergiocorreia/reghdfe>.

- Correia, Sergio. 2017. *Linear Models with High-Dimensional Fixed Effects: An Efficient and Feasible Estimator*. Technical report. <https://hdl.handle.net/10.2139/ssrn.3129010>.
- Correia, Sergio, Paulo Guimarães, and Tom Zylkin. 2020. “Fast Poisson Estimation with High-Dimensional Fixed Effects.” *The Stata Journal* 20 (1): 95–115. <https://doi.org/10.1177/1536867X20909691>.
- FixedEffectModels.jl Developers. 2026. “Fixedeffectmodels.jl: Fast Estimation of Linear Models with High Dimensional Categorical Variables.”. <https://github.com/FixedEffects/FixedEffectModels.jl>.
- Fong, David Chin-Lung, and Michael Saunders. 2011. “LSMR: An Iterative Algorithm for Sparse Least-Squares Problems.” *SIAM Journal on Scientific Computing* 33 (5): 2950–71. <https://doi.org/10.1137/10079687X>.
- Frisch, Ragnar, and Frederick V. Waugh. 1933. “Partial Time Regressions as Compared with Individual Trends.” *Econometrica* 1 (4): 387–401. <https://doi.org/10.2307/1907330>.
- Gao, Yuan, Rasmus Kyng, and Daniel A. Spielman. 2025. “AC(k): Robust Solution of Laplacian Equations by Randomized Approximate Cholesky Factorization.” *SIAM Journal on Scientific Computing*. <https://rasmuskyng.com/papers/GKS25.pdf>.
- Gaure, Simen. 2013. “OLS with Multiple High Dimensional Category Variables.” *Computational Statistics & Data Analysis* 66 : 8–18. <https://doi.org/10.1016/j.csda.2013.03.024>.
- Goldsmith-Pinkham, Paul. 2026. *Tracking the Credibility Revolution across Fields*. Technical report.
- Gremban, Keith D. 1996. “Combinatorial Preconditioners for Sparse, Symmetric, Diagonally Dominant Linear Systems.” Doctoral dissertation.
- Guimaraes, Paulo, and Pedro Portugal. 2010. “A Simple Feasible Procedure to Fit Models with High-Dimensional Fixed Effects.” *The Stata Journal* 10 (4): 628–49. <https://doi.org/10.1177/1536867X1101000406>.
- Head, Keith, and Thierry Mayer. 2014. “Gravity Equations: Workhorse, Toolkit, And Cookbook.” In *Handbook of International Economics*, edited by Gita Gopinath, Elhanan Helpman, and Kenneth Rogoff, vol. 4 of *Handbook of International Economics*. Elsevier. <https://doi.org/10.1016/B978-0-444-54314-1.00003-3>.
- Lovell, Michael C. 1963. “Seasonal Adjustment of Economic Time Series and Multiple Regression Analysis.” *Journal of the American Statistical Association* 58 (304): 993–1010. <https://doi.org/10.1080/01621459.1963.10480682>.
- Molitor, David. 2018. “The Evolution of Physician Practice Styles: Evidence from Cardiologist Migration.” *American Economic Journal: Economic Policy* 10 (1): 326–56. <https://doi.org/10.1257/pol.20160319>.
- PyFixest Developers. 2026. “Pyfixest: Fast High-Dimensional Fixed Effects Regression in Python.”. <https://github.com/py-econometrics/pyfixest>.
- Spielman, Daniel A., and Shang-Hua Teng. 2014. “Nearly Linear Time Algorithms for Preconditioning and Solving Symmetric, Diagonally Dominant Linear Systems.” *SIAM Journal on Matrix Analysis and Applications* 35 (3): 835–85. <https://doi.org/10.1137/090771430>.
- Stammann, Amrei. 2018. “Fast and Feasible Estimation of Generalized Linear Models with High-Dimensional K-Way Fixed Effects.”. <https://doi.org/10.48550/arXiv.1707.01815>.

- Toselli, Andrea, and Olof Widlund. 2005. *Domain Decomposition Methods: Algorithms and Theory*. Springer.
<https://doi.org/10.1007/b137868>.
- within Developers. 2026b. “Within Rust Crate.”. <https://crates.io/crates/within>.
- within Developers. 2026c. “Within-Py: Python Bindings for Within.”. <https://pypi.org/project/within-py/>.
- within Developers. 2026a. “Within: High-Performance Fixed-Effects Solver for Econometric Panel Data.”.
<https://github.com/py-econometrics/within>.
- within Developers. 2026d. “Withinr: R Bindings for Within.”. <https://py-econometrics.r-universe.dev/withinr>.
- Xu, Jinchao. 1992. “Iterative Methods by Space Decomposition and Subspace Correction.” *SIAM Review* 34 (4): 581–613. <https://doi.org/10.1137/1034116>.
- Yang, Mei, Gul Karaduman, and Ren-Cang Li. 2024. “Flexible Modified LSMR for Least Squares Problems.”.